Lab 6 — Counters, Clock/PLL & Shift Registers

Week 6 · Counter Applications (PWM, Timer) + Shift Registers (SIPO, Ring, LFSR) · Altera DE0 · CLOCK_50, KEY, SW ⟶ LEDG[9:0]

Name: _____	**Student ID:** _____	**Date:** _____	**Points:** _____

T/F — Exercise 1 — True or False — 10 pts

No	Statement	T/F	ถ้า F ⟶ เขียนคำตอบที่ถูกต้อง
1	Counter คือ register ที่เพิ่ม next-state logic (incrementer/decrementer) เข้าไป		
2	4-bit binary up counter นับได้ 32 ค่าที่แตกต่างกัน (0–31)		
3	Mod-10 counter ต้องใช้อย่างน้อย 5 bits เพราะ $clog2(10) = 5		
4	Clock skew คือขอบ clock ที่มาไม่ตรงเวลาในแต่ละรอบ ส่วน jitter คือ clock ถึง FF แต่ละตัวไม่พร้อมกัน		
5	Clock divider ที่ใช้ counter toggle สามารถเพิ่มความถี่ (multiply) ได้		
6	PLL สามารถทั้งคูณและหารความถี่ clock ได้ และให้ jitter ต่ำกว่า counter divider		
7	PWM generator สร้างได้จาก counter ตัวเดียวบวกกับ comparator หนึ่งตัว		
8	SIPO ใช้ในฝั่งส่งข้อมูลของ UART (UART TX) ส่วน PISO ใช้ในฝั่งรับ (UART RX)		
9	Ring counter แบบ 4-bit มี 4 states ส่วน Johnson counter แบบ 4-bit มี 8 states		
10	Barrel shifter เลื่อนข้อมูลได้หลายตำแหน่งภายใน 1 clock cycle ต่างจาก shift register ที่เลื่อนได้ทีละ 1 bit		

CODE — Exercise 2 — Complete the Up/Down Counter — 6 pts

```
module counter_updown (
  input  wire       clk, rst_n,
  input  wire       dir,
  input  wire       enable,
  output __(1)__ [3:0] count

always @(____(2)____ clk) begin
  if (____(3)____)
    count <= 4'b0;
  else if (____(4)____)
    if (dir)
      count <= count + __(5)__;
    else
      count <= count - __(6)__;
end

endmodule
```

ช่อง	คำตอบ	เพราะอะไร?
(1)		
(2)		
(3)		
(4)		
(5)		
(6)		

DEBUG	Exercise 3 — Find the 4 Bugs in Clock Divider	8 pts

```
1   module clk_divider (
2    input  wire  clk_50m, rst_n,
3    output wire  clk_1hz
4   );
5   reg [23:0] divider;
6   always @(posedge clk_50m) begin
7     if (!rst_n) begin
8       divider = 0;
9         clk_1hz = 0;
10      end else if (divider == 25_000_000) begin    // BUG 4
11        divider <= 0;
12        clk_1hz <= ~clk_1hz;
13      end else
14        divider <= divider + 1;
15    end
16  endmodule
```

Bug	บรรทัด	ผิดอย่างไร	แก้เป็น
1			
2			
3			
4			

CALC	Exercise 4 — PLL Calculation	10 pts

สูตร PLL: f_out = (f_in × N) / (M × K) M = input divider, N = feedback multiplier, K = output divider

Worked Example: ต้องการ 108 MHz (VGA) จาก f_in = 50 MHz

Step 1: หา ratio = 108/50 = 2.16 = 54/25 ⟶ N=54, M=25

Step 2: เช็ค f_VCO = 50 × 54/25 = 108 MHz ⟶ ต่ำกว่า VCO min 300 MHz ✗

Step 3: คูณ N ด้วย 5 ⟶ N=270, M=25, K=5 ⟶ f_VCO = 50×270/25 = 540 MHz ✓ (อยู่ใน 300–600)

Step 4: f_out = f_VCO / K = 540/5 = 108 MHz ✓ ได้ตามเป้าหมาย

f_in	f_out ที่ต้องการ	M	N	f_VCO	K	f_out จริง
50 MHz	100 MHz					
50 MHz	25 MHz					
50 MHz	75 MHz					

f_in	f_out ที่ต้องการ	M	N	f_VCO	K	f_out จริง
50 MHz	40 MHz					
50 MHz	70 MHz (XGA)					

CONCEPT	Exercise 5 — Shift Register Types & Code	10 pts

(a) จับคู่ประเภท shift register กับการใช้งาน — เขียน SISO / SIPO / PISO / PIPO ลงในช่อง:

การใช้งาน	ประเภท	เหตุผล
UART TX — ส่งข้อมูล 8 bit ออกทางสายเดียว		
ชิป 74HC595 — ใช้ 3 ขา MCU คุม LED 8 ดวง		
Delay line — หน่วงสัญญาณ 4 clock cycles		
Register ธรรมดาที่โหลดและอ่านพร้อมกันทุก bit		

(b) เติมช่องว่างให้ครบ — ใช้ concatenation operator { } ในการเลื่อนบิต:

```
// Shift LEFT: bit ใหม่เข้าที่ LSB, bit บนสุดหลุดออก
q <= { ____(1)____ , ser_in };

// Shift RIGHT: bit ใหม่เข้าที่ MSB, bit ล่างสุดหลุดออก
q <= { ser_in , ____(2)____ };

// Rotate LEFT: bit บนสุดวนกลับเข้า LSB
q <= { q[6:0] , ____(3)____ };

// Ring counter 4-bit: 1000 -> 0100 -> 0010 -> 0001 -> 1000
q <= { _________ , _________ };      // (4)
```

ช่อง	คำตอบ
(1)	
(2)	
(3)	
(4)	

LAB	Exercise 9 — Design & Verification	10 pts

โจทย์: ใช้ ALTPLL ตัวเดียวสร้าง 3 ความถี่พร้อมกัน จาก CLOCK_50 แล้วแจกจ่ายให้แต่ละส่วนของวงจรตามความเหมาะสม — แสดงผลบน LEDG[9:0]

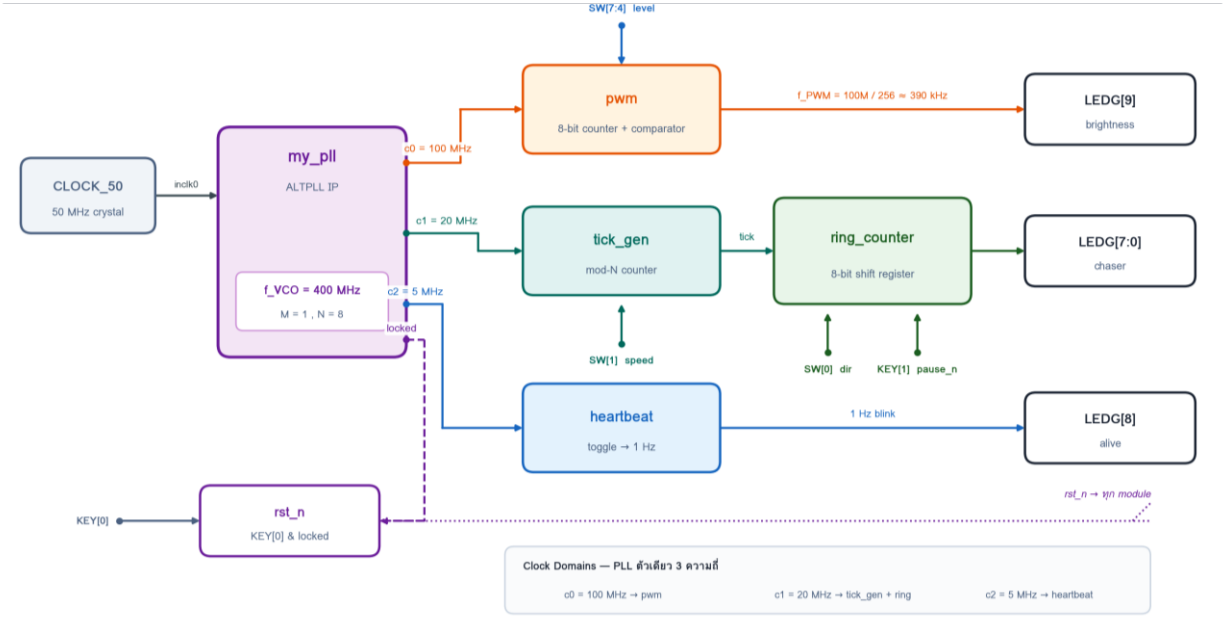
Specification:

Clock	ความถี่	ป้อนให้	หน้าที่ / Output
c0	100 MHz	pwm	หรี่ไฟ LEDG[9] ตาม SW[7:4] (16 ระดับ)
c1	20 MHz	tick_gen ⟶ ring_counter	ไฟวิ่ง LEDG[7:0] · SW[0]=ทิศทาง · SW[1]=ความเร็ว
c2	5 MHz	heartbeat	LEDG[8] กระพริบ 1 Hz (บอกว่าระบบยังทำงาน)
locked	—	reset logic	rst_n = KEY[0] & locked (รอ PLL นิ่งก่อน)

ปุ่ม / สวิตช์	หน้าที่
KEY[0]	Reset — ring counter กลับตำแหน่งแรก (1000_0000)
KEY[1]	Pause — กดค้างเพื่อหยุดไฟวิ่งชั่วคราว
SW[0]	ทิศทาง: 1 = วิ่งซ้าย, 0 = วิ่งขวา
SW[1]	ความเร็ว: 0 = 2 Hz (ช้า), 1 = 10 Hz (เร็ว)
SW[7:4]	ระดับความสว่างของ LEDG[9] (0–15)

#	ไฟล์	clock ที่ใช้	หน้าที่
1	my_pll.v	—	สร้างจาก ALTPLL IP Wizard (3 outputs)
2	pwm.v	c0 100 MHz	หรี่ไฟ LEDG[9]
3	tick_gen.v	c1 20 MHz	สร้าง tick 2 Hz / 10 Hz
4	ring_counter.v	c1 20 MHz	ไฟวิ่ง LEDG[7:0]
5	heartbeat.v	c2 5 MHz	กระพริบ LEDG[8] 1 Hz
6	DE0_led_effects.v	ทั้งหมด	top-level เชื่อมทุก module

Block Diagram:



ไฟล์ 1: my_pll.v — ตั้งค่าใน ALTPLL IP Wizard

Tools ⟶ IP Catalog ⟶ Basic Functions ⟶ Clocks; PLLs and Resets ⟶ PLL ⟶ ALTPLL (ตั้งชื่อ my_pll)

ไฟล์ 2: pwm.v — clock domain c0 (100 MHz)

```
module pwm (
```

```
  input  wire       clk,
  input  wire       rst_n,
  input  wire [3:0] level,
  output wire       pwm_out
);

  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
endmodule
```

ไฟล์ 3: tick_gen.v — clock domain c1 (20 MHz)

```
module tick_gen #(
  parameter integer CNT_SLOW = _____________________________
  parameter integer CNT_FAST = _____________________________
)(
  input  wire  clk,
  input  wire  rst_n,
  input  wire  speed,
  output wire  tick
);
  reg [23:0] cnt;

  wire [23:0] limit = ____________________________________

  always @(posedge clk)
    if (!rst_n)
       cnt <= ________________________________________
    else if ________________________________________
       cnt <= ________________________________________
    else
       cnt <= ________________________________________

  assign tick = ________________________________________
endmodule
```

ไฟล์ 4: ring_counter.v — clock domain c1 (20 MHz)

```
module ring_counter (
  input  wire       clk, rst_n,
  input  wire       tick,
  input  wire       dir,
  input  wire       pause_n,
  output reg  [7:0] led
);
  always @(posedge clk) begin
    if (!rst_n)
      led <= 8'b_____________;
    else if (_____________)
      led <= led;
    else if (________) begin
      if (dir) led <= { _______ , ______ };
      else     led <= { _______ , ______ };
    end
  end
endmodule
```

ไฟล์ 5: heartbeat.v — clock domain c2 (5 MHz)

```
module heartbeat (
  input  wire  clk,
  input  wire  rst_n,
  output reg   led
);
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
  ____________________________________________________________________________
endmodule
```

ไฟล์ 6: DE0_led_effects.v — top-level เชื่อมทุก module และทุก clock domain

```
module DE0_led_effects (
 input wire CLOCK_50,
 input wire [2:0] KEY,
 input wire [9:0] SW,
 output wire [9:0] LEDR
);

 wire clk_100m, clk_20m, clk_5m, locked;

 my_pll u_pll (
   .inclk0 (CLOCK_50),
   .c0 (clk_100m),
   .c1 (clk_20m),
   .c2 (clk_5m),
   .locked (locked)
 );

 wire rst_n = KEY[0] & locked;

 pwm u_pwm (
   .clk (clk_100m),
   .rst_n (rst_n),
   .level (SW[7:4]),
   .pwm_out (LEDG[9])
 );

 wire tick;
 wire [7:0] led_bus;

 tick_gen u_tick (
  .clk (clk_20m),
  .rst_n (rst_n),
  .speed (SW[1]),
  .tick (tick)
 );

 ring_counter u_ring (
   .clk (clk_20m),
  .rst_n (rst_n),
  .tick (tick),
  .dir (SW[0]),
```

```
  .pause_n (KEY[1]),
  .led (led_bus)
 );

 assign LEDR[7:0] = led_bus;

 heartbeat u_hb (
  .clk (clk_5m),
  .rst_n (rst_n),
  .led (LEDR[8])
 );
endmodule
```

รายการ	ค่าที่ได้
Total logic elements	
Total registers	
Total PLLs	
Fmax ของ clk_100m domain	
Fmax ของ clk_20m domain	
Fmax ของ clk_5m domain	

ทดสอบบนบอร์ดจริง — ทำเครื่องหมาย ✓ เมื่อผ่าน

การทดสอบ	ผลที่ควรได้	✓
เปิดเครื่อง	LEDR[8] กระพริบสม่ำเสมอ 1 ครั้ง/วินาที (PLL lock + c2 ทำงาน)	
กด KEY[0]	LEDR[7] ติดดวงเดียว (1000_0000)	
SW[0] = 1 / 0	ไฟวิ่งซ้าย / ขวา แล้ววนกลับ ไม่หายไป	
SW[1] = 0 ⟶ 1	ความเร็วเพิ่มจาก 2 Hz เป็น 10 Hz	
กด KEY[1] ค้าง	ไฟวิ่งหยุด แต่ LEDR[8] ยังกระพริบต่อ (คนละ clock domain)	
SW[7:4] = 0000 ⟶ 1111	LEDR[9] สว่างขึ้น 16 ระดับ ไม่กระพริบ	